

PeerChat — Secure Decentralized P2P Messaging System

PeerChat is a decentralized peer-to-peer messaging platform written in Python. Each node acts as both a client and server, enabling direct encrypted communication without relying on a centralized backend. **Identity is cryptographically derived from RSA keys, ensuring that each user is uniquely identifiable and resistant to spoofing.**

The system supports:

- Context-aware private and global messaging
 - Automatic peer discovery across LAN and local environments
 - RSA-based challenge-response authentication
 - Persistent message history with timestamps
 - File transfer via Base64-encoded packets
 - Modernized, smooth-corner PyQt6 user interface with localized startup configuration view
-

Releases & Downloads



v1.0.0 — The Initial Decentralized Release

This marks the first stable release of PeerChat, moving the project from a command-line proof-of-concept to a fully-realized desktop application with a modern graphical interface.

Binary Downloads

- **Windows (x64):** `peerchat.exe`



Note on Build Generation: If compiling from source using PyInstaller, use the following command to match the official release naming convention:

```
pyinstaller --onefile --windowed --name=peerchat --  
icon=assets/peerchat.ico main.py
```

Features

Networking & Data Transfer

- Fully decentralized peer-to-peer architecture
- TCP socket communication using newline-delimited JSON packets
- Automatic peer discovery through bootstrap peers and gossip-based routing
- Multi-peer simultaneous communication with continuous receive loops
- Robust file transfer using Base64 encoding to ensure cross-platform integrity
- Network resilience through retry cooldowns and peer revalidation

Connectivity Note: At least one bootstrap peer must be reachable for network discovery to begin.

Security

- RSA Public/Private key cryptography for identity verification
 - Cryptographic Peer IDs derived from public keys
 - Challenge-response authentication prevents impersonation
 - No passwords or centralized identity storage
 - Secure key storage in local `keys/` directory
-

Messaging & Sharing

- Direct private messaging between peers
- Global broadcast messaging to all connected peers
- File attachments with automatic download handling
- JSON-based message protocol for structured routing
- Local persistence of all messages and file references
- Timestamped message history for accurate chronological reconstruction
- Received files are automatically saved to the `downloads/` directory

GUI

The interface has been fully updated to a sleek, modern visual aesthetic based on the dark palette theme.

Configuration Window (Startup Screen)

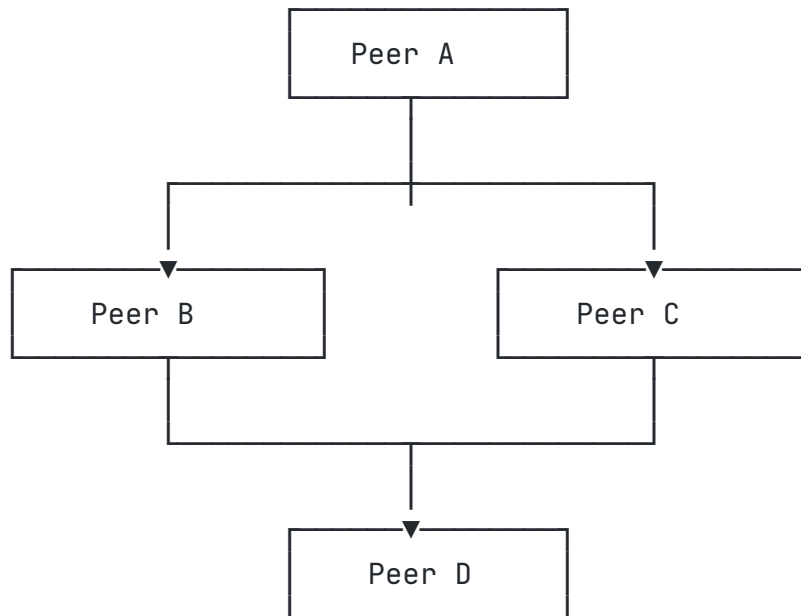
- **Glitch-Free UI Rendering:** Form fields use locked structural bounds and anti-flicker layout placeholders to prevent font shaking or popping on app initialization.
- **Modern Geometric Aesthetics:** Features soft floating container cards with rounded corners (24px border-radius).
- **Validation Subsystem:** Handles improper or unassigned form fields locally before spinning up network socket resources.

Main Chat Window

- **Enhanced Peer List:** Bold, high-contrast typography displaying active peer statuses against rich background blocks for excellent visibility.
- **Integrated Control Panels:** The attachment clip button () and message entry boxes are seamlessly styled side-by-side inside custom inner frames.
- **Real-Time Indicators:** Real-time online/offline peer visibility tracking for currently connected peers.

Architecture

The PeerChat network operates as a decentralized mesh where each node maintains direct connections with its peers.



Each peer node integrates the following core components:

- Configuration interface launcher or CLI interceptor
- Listener server & outgoing client connector
- Message handler & peer discovery engine
- Authentication layer (RSA challenge-response)
- Local SQLite database with timestamped history
- PyQt6 frontend with real-time updates

Authentication Flow

PeerChat uses cryptographic verification instead of passwords to ensure secure and decentralized identity management.

1. **Connect:** Establish a TCP connection between peers.
2. **Exchange public keys:** Exchange identity packets containing RSA public keys.
3. **Generate random challenge:** One peer sends a random string to the other.
4. **Sign challenge:** The recipient signs the challenge with their private key.

5. **Verify signature:** The sender verifies the signature using the recipient's public key.
 6. **Authenticated Communication:** Secure messaging begins upon successful verification.
-

Message Protocol

Chat Packet (Direct / Global)

```
{
  "type": "chat",
  "data": {
    "sender": "Alice-a1b2c3d4",
    "recipient": "Bob-e5f6g7h8",
    "message": "Hello, Bob!"
  }
}
```

If `recipient` is `null`, the message is treated as a global broadcast.

File Transfer Packet

```
{
  "type": "file_transfer",
  "data": {
    "sender": "Alice-a1b2c3d4",
    "recipient": "Bob-e5f6g7h8",
    "file_name": "document.pdf",
    "payload": "BASE64_ENCODED_DATA..."
  }
}
```

Folder Structure

```
peerchat/
├── main.py           # Entry point (GUI or CLI)
├── config.py         # Runtime settings
├── assets/           # Static UI resources
│   ├── logo.ico
│   ├── logo.svg
│   ├── peerchat.ico
│   └── peerchat.svg
├── keys/             # RSA key storage
├── downloads/        # Received file attachments
├── gui/
│   ├── app.py        # GUI launcher
│   ├── config_window.py # Configuration screen
│   ├── chat_window.py # Chat rendering logic
│   └── signals.py     # Event bus system
├── network/
│   ├── server.py     # Handshake & routing
│   ├── client.py     # Sending & transfers
│   └── discovery.py  # Peer discovery
├── security/
│   ├── keys.py       # Key management
│   └── crypto.py     # Cryptographic utilities
└── storage/
    └── database.py   # SQLite persistence
```

Running PeerChat from Source

Install Dependencies

```
pip install pyqt6 cryptography
```

Running via Configuration GUI

Launch PeerChat without flags to use the high-contrast setup screen:

```
python main.py
```

Running via Classic Terminal Fallback

The system retains full backward compatibility for command-line arguments:

1. Start Bootstrap Nodes:

```
python main.py 9000 Bootstrap_node  
python main.py 9001 Bootstrap_node_1  
python main.py 9002 Bootstrap_node_2
```

2. Start User Peers:

```
python main.py <Port> <Username>
```

Bootstrap Peers Configuration

PeerChat uses default bootstrap peers to initialize network discovery, defined in `network/discovery.py`.

Local Testing (Single Machine)

```
BOOTSTRAP_PEERS = [  
    ("127.0.0.1", 9000),  
    ("127.0.0.1", 9001),  
    ("127.0.0.1", 9002)  
]
```

LAN Setup

For local network deployment, replace IPs with the LAN addresses of your machines:

```
BOOTSTRAP_PEERS = [  
    ("192.168.1.2", 9000),  
    ("192.168.1.3", 9001)  
]
```

Planned Improvements

- Message delivery status indicators (✓ / ✓✓)
- Unread message counters
- Typing indicators
- File transfer progress bars

License

MIT License. See `LICENSE` for details.